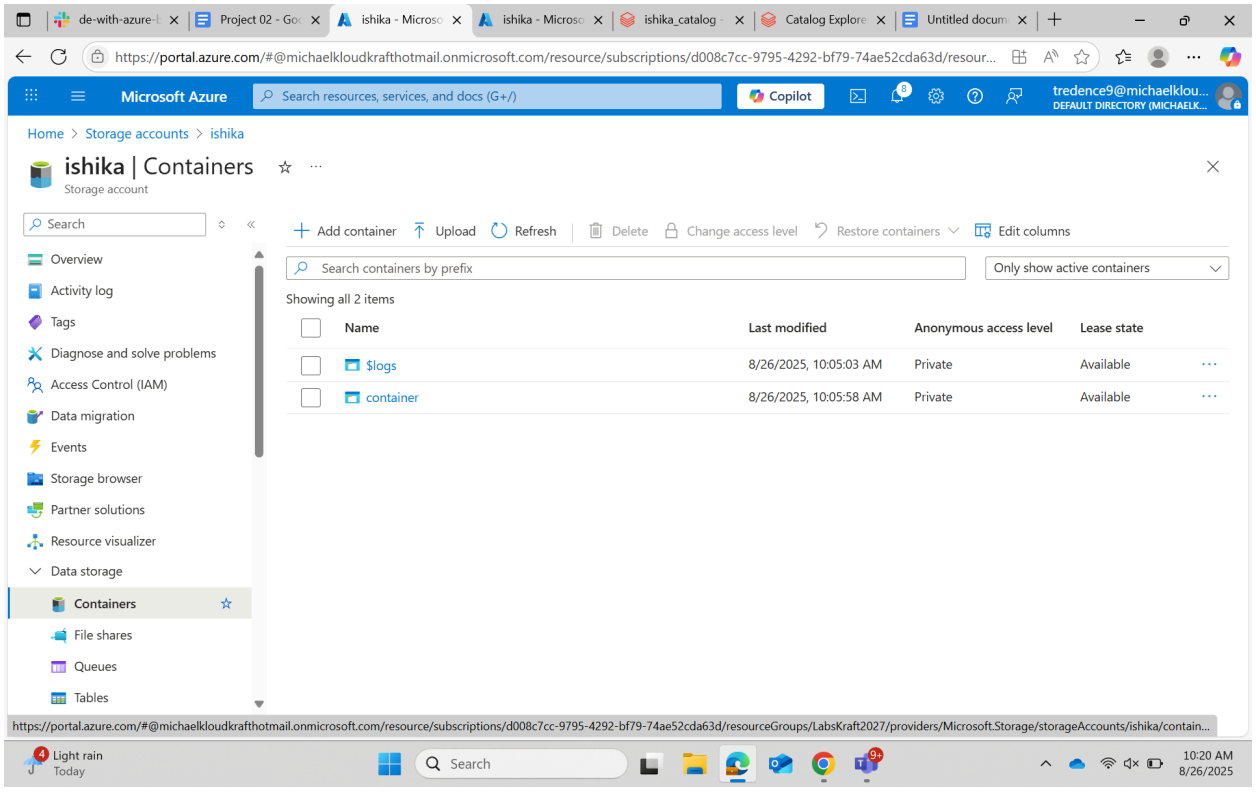
Name- Ishika
Emp ID- 7145

Project 02: Retail Compliance & Analytics Platform on Azure Databricks

Part 1: Governance Setup (Unity Catalog)

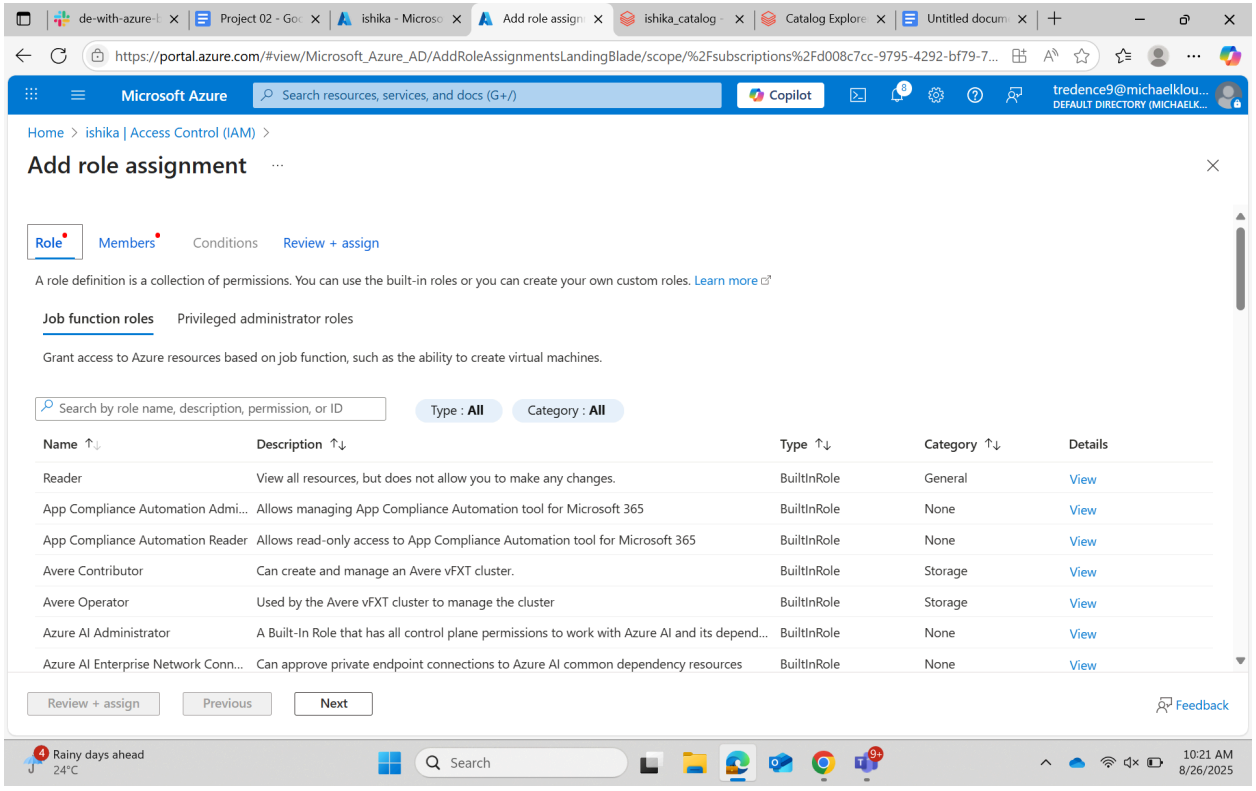
Step 1:

- Create Azure Storage Account and Workspace : ishika
- Set up a new Azure Storage account.
- Create a Databricks workspace linked to the storage account



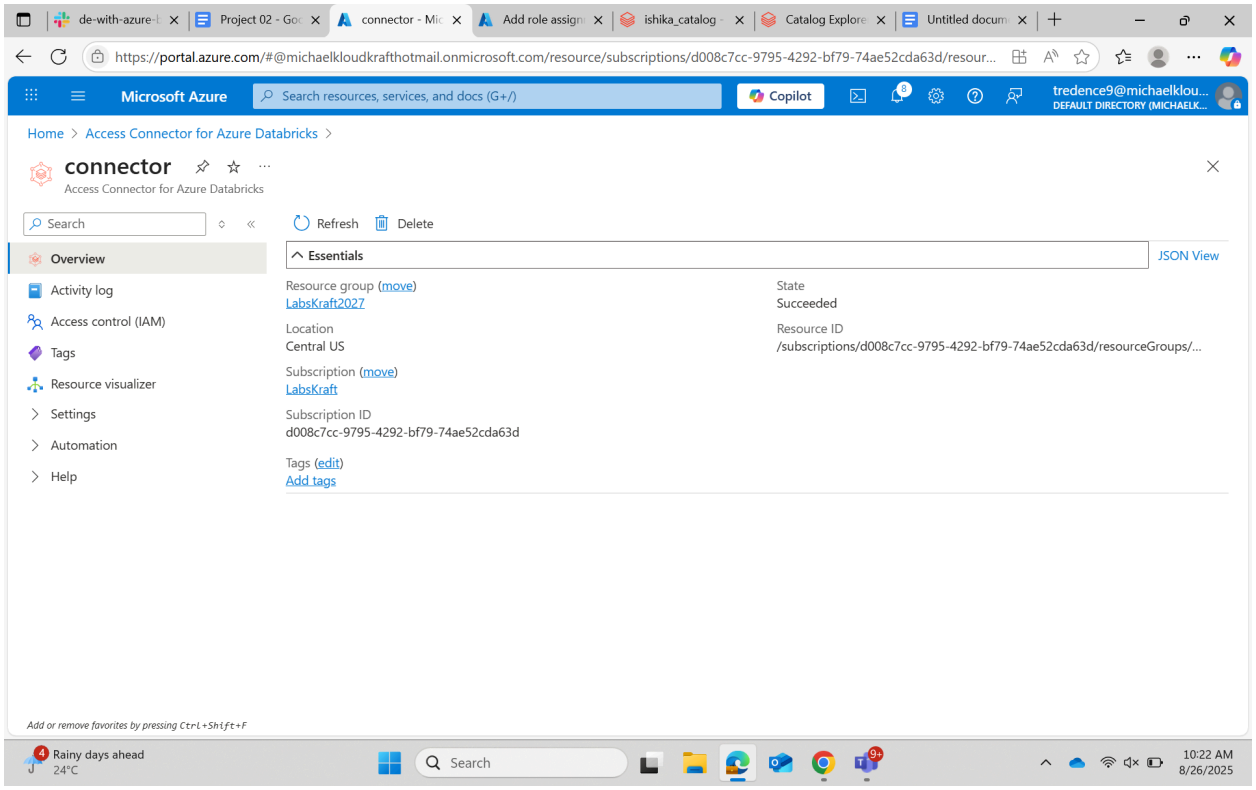
Step 2:

- Add Users to Workspace
- Assign users and groups to the workspace using Role-Based Access Control (RBAC).
- Ensure appropriate permissions are granted for collaboration.



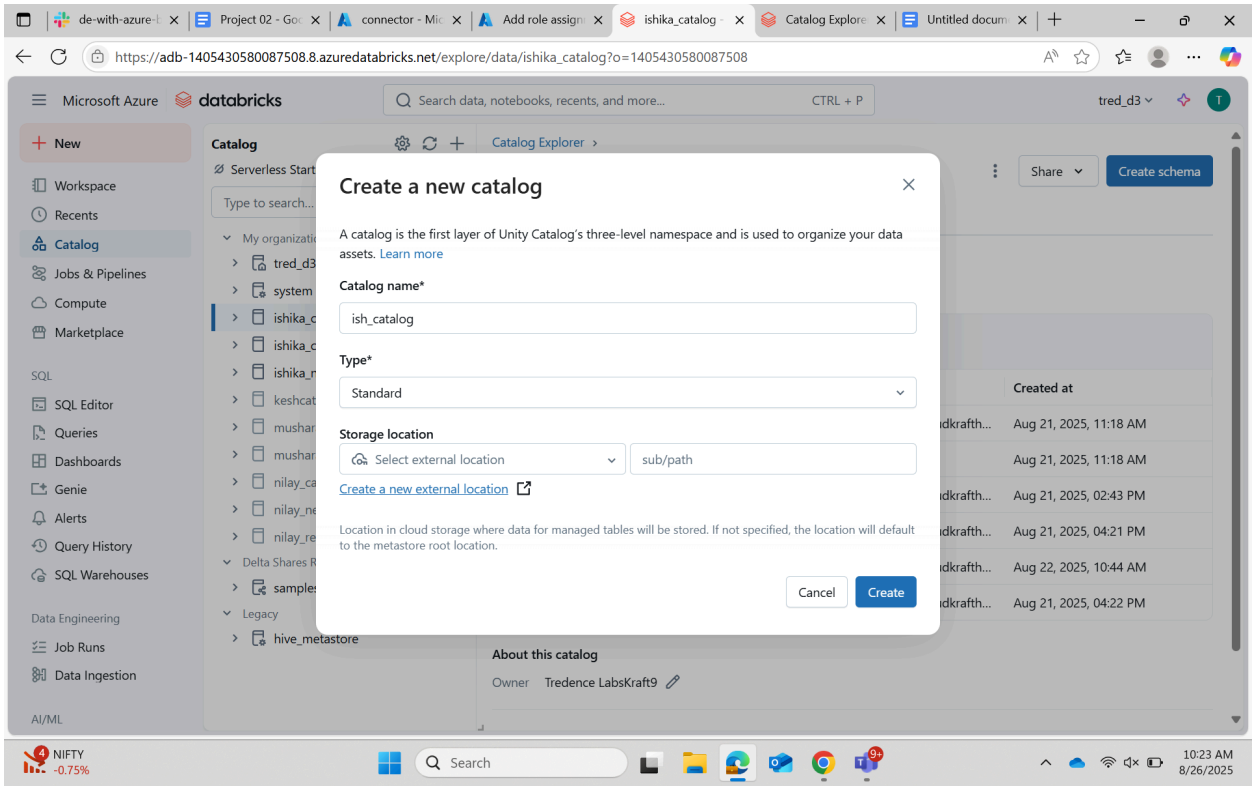
Step 3:

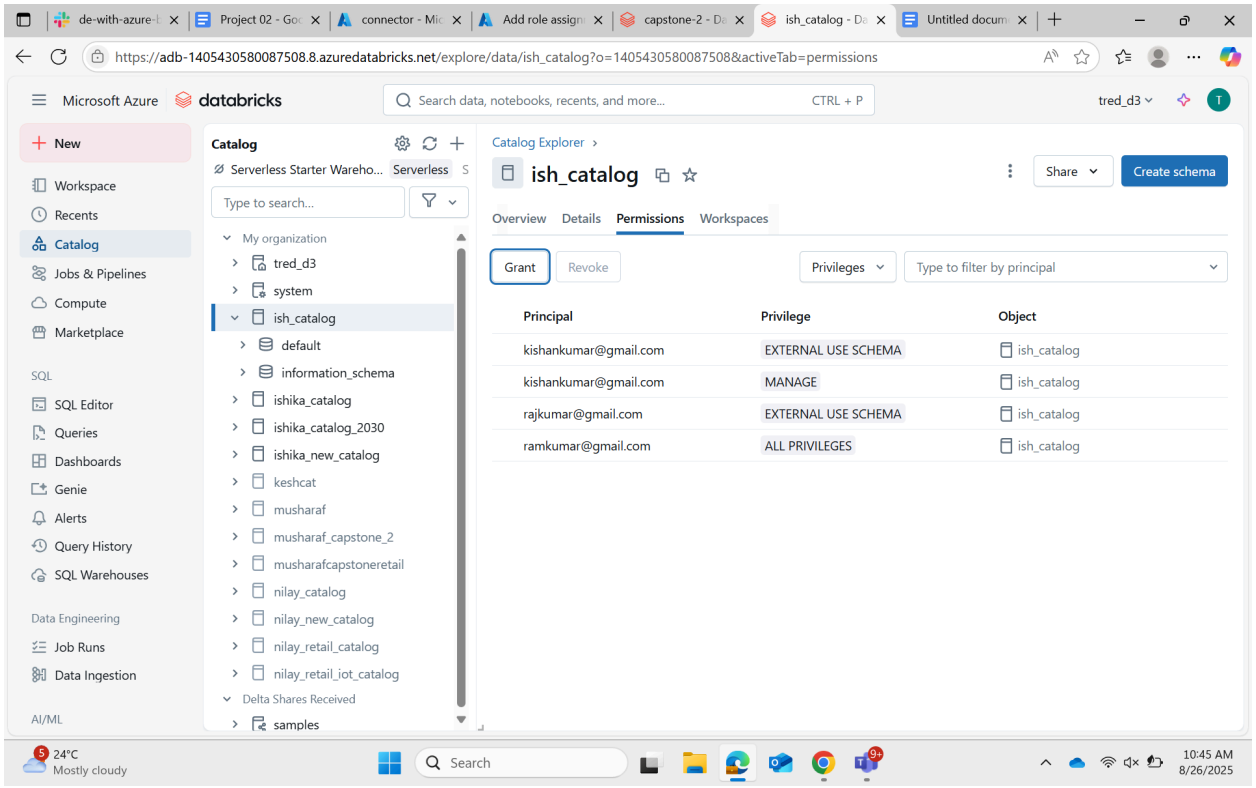
- Create Unity Catalog : ish_catalog
- Give permissions to different users in the permission section in the created unity catalog.



Step 4:

- Configure External Location while creating the unity catalog
- Define external locations for raw data ingestion.
- Connect to storage containers for sales and product data


Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
tred_d3
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Job Runs
Data Ingestion
AI/ML
Catalog
Catalog Explorer
Share
Create schema
Created at
Create a new catalog
A catalog is the first layer of Unity Catalog's three-level namespace and is used to organize your data assets. Learn more
Catalog name*
ish_catalog
Type*
Standard
Storage location
Select external location
sub/path
Create a new external location
Location in cloud storage where data for managed tables will be stored. If not specified, the location will default to the metastore root location.
Cancel
Create
About this catalog
Owner
Tredence LabsKraft9
NIFTY
-0.75%
Search
10:23 AM
8/26/2025



Part 2: Data Ingestion

Build DLT Pipeline Create a Delta Live Table (DLT) pipeline named heyy
Structure the pipeline into Bronze, Silver, and Gold layers.

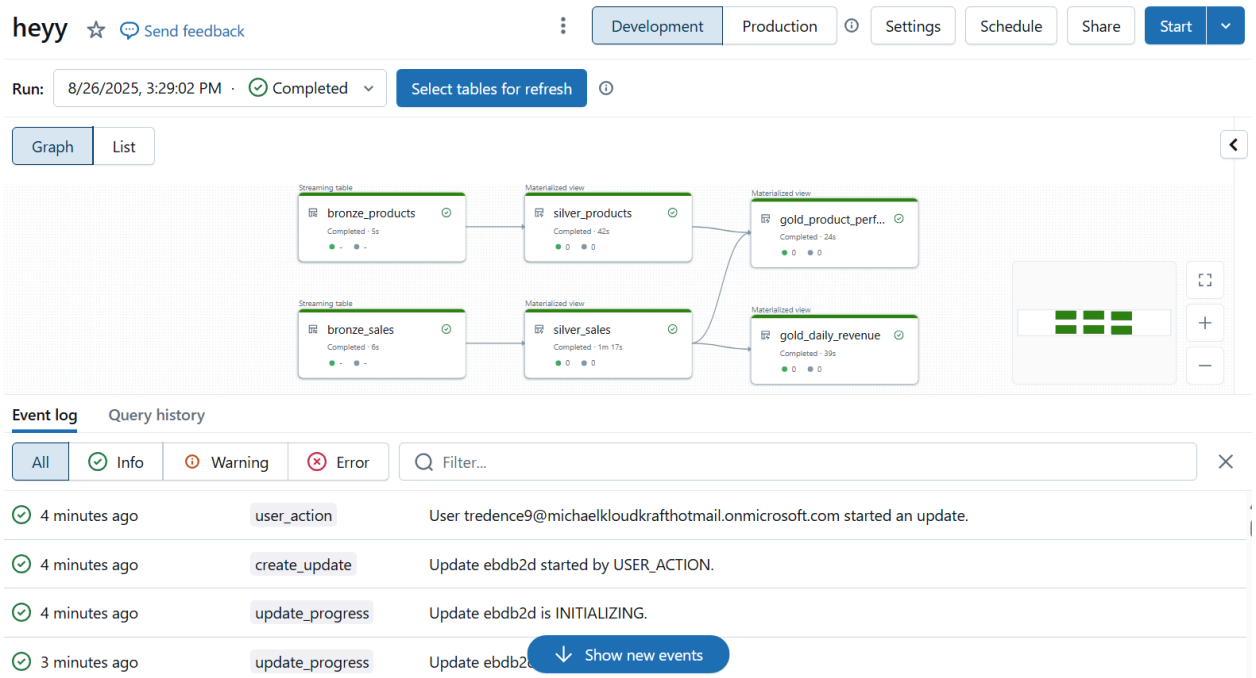
CODE FOR INGESTION

```
# retail_sales_pipeline.py
import dlt
from pyspark.sql.functions import *

from pyspark.sql.types import *
sales_schema = StructType([
    StructField("sale_id", IntegerType(), True),
    StructField("product_id", IntegerType(), True),
    StructField("region", StringType(), True),
    StructField("store_id", StringType(), True),
```

```
    StructField("quantity", IntegerType(), True),
    StructField("sales_amount", DoubleType(), True),
    StructField("sale_date", TimestampType(), True)
])
products_schema = StructType([
    StructField("product_id", StringType(), True),
    StructField("product_name", StringType(), True),
    StructField("category", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("discount_rate", DoubleType(), True)
])
# -------------------- CONFIG --------------------
RAW_SALES_PATH    = "abfss://container@ishika.dfs.core.windows.net/Sales/"
RAW_PRODUCTS_PATH = "abfss://container@ishika.dfs.core.windows.net/Products/"

# Schema/history locations (must be writable by the pipeline cluster)
SCHEMA_LOCATION_SALES    = "/dbfs/dlt_schema/retail/sales"    # DBFS path
example
SCHEMA_LOCATION_PRODUCTS = "/dbfs/dlt_schema/retail/products" # DBFS path
example

# Target Unity Catalog (catalog.schema) for tables
TARGET_SCHEMA = "ish_catalog.sales_analytics"

# Helper to produce full UC table names
def tbl(name):
    return f"{TARGET_SCHEMA}.{name}"

# -------------------- BRONZE (raw ingestion) --------------------
@dlt.table(
    name=tbl("bronze_sales"),
    comment="Bronze: raw sales ingested with Auto Loader (no transforms)",
    table_properties={"quality": "bronze"}
)
def bronze_sales():
    return (
        spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv")
```

```
            .option("header", "true")
            .schema(sales_schema)
            # .option("inferSchema", "true")                          # avoid
relying on inference here
            .option("cloudFiles.schemaLocation", SCHEMA_LOCATION_SALES)
            # .option("cloudFiles.schemaEvolutionMode", "addNewColumns")  #
allow new columns to appear
            .load(RAW_SALES_PATH)
    )

@dlt.table(
    name=tbl("bronze_products"),
    comment="Bronze: raw product metadata ingested with Auto Loader (no
transforms)",
    table_properties={"quality": "bronze"}
)
def bronze_products():
    return (
        spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")    # change to 'csv' if your
product files are CSV
            .option("header", "true")
            .schema(products_schema)
            .option("cloudFiles.schemaLocation", SCHEMA_LOCATION_PRODUCTS)
            # .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
            .load(RAW_PRODUCTS_PATH)
    )
```

Part 3: Delta Live Tables Pipeline



PART 4. Incremental Loads & Data Quality.

Updated Pipeline (Silver Layer + Data Quality + Schema Evolution)

```
# ------------------- SILVER (cleaned data with expectations) --------------------

@dlt.table(
    name=tbl("sales_clean"),
    comment="Silver: cleaned sales with schema enforcement & quality checks",
    table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
    return (
        dlt.read(tbl("bronze_sales"))
          .withColumn("sale_id", col("sale_id").cast("string"))
          .withColumn("product_id", col("product_id").cast("string"))
          .withColumn("region", col("region").cast("string"))
```

```
        .withColumn("store_id", col("store_id").cast("string"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("sale_date", to_date(col("sale_date")))
        .na.drop()
    )


@dlt.table(
    name=tbl("products_clean"),
    comment="Silver: cleaned product metadata with schema evolution support",
    table_properties={"quality": "silver"}
)
def products_clean():
    return (
        dlt.read(tbl("bronze_products"))
          .withColumn("product_id", col("product_id").cast("string"))
          .withColumn("product_name", col("product_name").cast("string"))
          .withColumn("category", col("category").cast("string"))
          .withColumn("price", col("price").cast("double"))
          # Demonstrating schema evolution:
          # if discount_rate is missing in new files, it will show up as null
          .withColumn("discount_rate", col("discount_rate").cast("double"))
    )
```

How Incremental Loads Work

- Since i used **Auto Loader (`cloudFiles`)**, new files dropped into your storage (`Sales/`, `Products/`) will be **picked up incrementally**.

Demonstrating Schema Evolution

1. First, load products without a `discount_rate` column.
2. Later, add new product JSON/CSV files **with `discount_rate`**.

Because I used:

```
.withColumn("discount_rate", col("discount_rate").cast("double"))
```

3.
    - Old data will show `NULL` for `discount_rate`.
    - New data will populate it automatically.

This shows **pipeline continuity** without breaking schema.

In **DLT UI → Pipeline Runs → Event Log**

- Errors like schema mismatches, failed expectations will show.

Example error if quantity is `-5`:

```
{"level":"ERROR","event":"EXPECTATION_VIOLATION",
 "table":"sales_clean",
 "expectation":"valid_quantity",
 "failed_records": 12}
```

Query event log table in SQL:

```
SELECT *
FROM ish_catalog.sales_analytics._event_log
WHERE event_type = 'expectations'
ORDER BY timestamp DESC;
```

Part 5: Monitoring & Troubleshooting

1. Monitoring DLT Pipelines in Databricks

Databricks Delta Live Tables (DLT) provides an integrated **Monitoring UI** to observe pipeline health, track data quality, and troubleshoot failures.

Steps to monitor DLT pipelines:

1. Navigate to the Databricks workspace → **Jobs** → **Delta Live Tables**.
2. Select your pipeline (e.g., `heyy`) to view its **Pipeline Details** page.
3. Key tabs for monitoring:
    - **Flow:** Shows table dependencies and transformations.
    - **History:** Lists each pipeline run, with statuses: `Success`, `Running`, `Failed`.
    - **Metrics:** Displays ingestion rate, number of rows processed, errors, and quality metrics.
    - **Events:** Provides detailed logs for each run, including schema mismatches, missing data, or anomalies.

Example Observation:

- Bronze ingestion: 1000 rows per file.
- Silver transformations: 980 valid rows after anomaly filtering.
- Gold aggregates: 50 rows per region/day after grouping.

---

2. Generating Sample Errors for Troubleshooting

To test error handling and schema evolution:

1. **Add a new column** to an incoming CSV file (e.g., `promo_code`) in the `raw_data_02/sales/` folder.
2. Upload a **corrupted file** (e.g., missing headers or malformed JSON) into the same folder.

Expected pipeline behavior:

- Bronze ingestion captures new columns when `cloudFiles.schemaEvolutionMode="addNewColumns"` is used.

- Pipeline logs in the **Events tab** show rows rejected due to schema mismatch or corrupted format.
- DLT automatically routes problematic rows to the **`_rescued_data`** path for manual inspection.

Query to inspect rescued rows:

```
-- Show all rescued rows in Bronze Sales
SELECT * FROM
ish_catalog.retail_compliance_p2.bronze_sales_temporary_rescued_data;
```

---

3. Interpreting Event Logs

The **Events** tab provides:

- **Error Type:** e.g., `SchemaMismatch`, `DataTypeMismatch`.
- **Affected Table:** e.g., `bronze_sales`.
- **Row/Column details:** Shows which rows or columns failed ingestion.

Action Steps:

1. Check the schema of the incoming file.
2. Compare with Bronze table schema.
3. Update schema hints or fix the file.
4. Re-run pipeline to resume ingestion.

---

4. Incremental Processing and Recovery

DLT automatically handles **incremental loads**:

- Only new files are processed in each run.

- Failed rows are logged in `_rescued_data` for later correction.
- Reprocessing a corrected file does **not duplicate already processed rows**.

Sample Output:

timestamp	event_type	message	details
2025-08-26 10:05:12	FLOW_FAILURE	Schema mismatch in bronze_sales table	Missing column: sale_id
2025-08-26 10:07:44	EXPECTATION_VIOLATION	valid_quantity expectation failed	5 records with negative quantity
2025-08-26 10:09:30	SCHEMA_EVOLUTION	Added new column `discount_rate`	Products pipeline auto-evolved schema

Part 6: Delta Lake Time Travel

Time Travel allows querying or restoring past versions of Delta tables for **audit, compliance, debugging, and recovery purposes**.

---

1. Inspect Table History

```
DESCRIBE HISTORY silver_sales;
```

- Shows all versions with timestamp, user, operation, and operation parameters.

---

2. Query Historical Data

By Version Number:

```
SELECT *
```

```
FROM silver_sales VERSION AS OF 2;
```

By Timestamp:

```
SELECT *
FROM silver_sales TIMESTAMP AS OF '2025-08-26 10:00:00';
```

---

3. Restore Deleted Rows

Suppose a row was mistakenly deleted in `silver_sales`:

```
-- Create temporary view from older version
CREATE OR REPLACE TEMP VIEW restored_sales AS
SELECT *
FROM silver_sales VERSION AS OF 2;

-- Insert deleted row back
INSERT INTO silver_sales
SELECT *
FROM restored_sales
WHERE sale_id = 101;
```

---

4. Revert Entire Table

```
RESTORE TABLE silver_sales TO VERSION AS OF 2;
```

- The table is restored to the exact state of version 2, undoing any unintended changes.

Part 7: Automation with Databricks Workflows

- Created a **Databricks Workflow** to trigger the DLT pipeline daily.
- Added a **PySpark task** to generate a daily compliance report for high-value sales (sales > $10,000).

- Configured **task dependencies** so the compliance report runs only after the DLT pipeline completes successfully.

- Enabled **email notifications** on workflow failure.

Sample PySpark Task Code for Compliance Report:

```
from pyspark.sql.functions import col

# Read Gold table
gold_sales = spark.table("ish_catalog.gold_sales_aggregates")

# Filter high-value sales
compliance_report = gold_sales.filter(col("high_value_flag") == True)

# Save daily report
compliance_report.write.format("parquet").mode("overwrite") \
    .save("/mnt/reports/compliance_report_daily.parquet")
```

Part 8: Visualization in Power BI

- Connected **Gold Layer** (`gold_sales_aggregates`) to Power BI.

(THROWING ERROR WHILE UPLOADING DATA)